

第五章 5.2–5.3 图文试改

5.2 单算子：分块、布局与实际访存

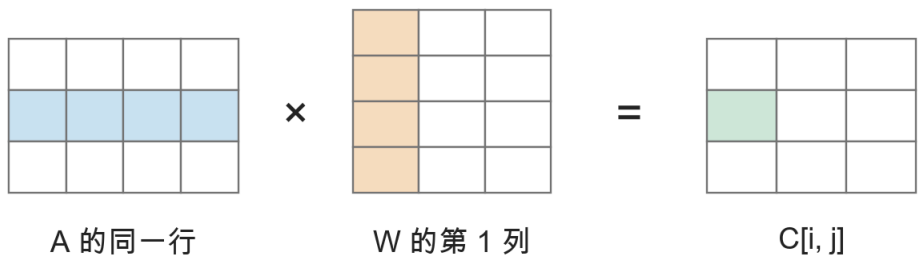
矩阵乘中，同一个输入元素参与多个输出的计算。同一个权重元素又被多行输入使用。如果将读入的数据保留在局部存储中，就能用一次读取完成多次乘加。要实现这种复用，需要同时考虑循环顺序、局部存储容量和任务分配。

5.2.1 矩阵形状与重复读取

以下用 A 表示章首的输入 X ，投影写作 $C = AW$ ，其中 $M = 1024, K = 4096, N = 12288$ 。 A 、 W 、 C 采用 BF16（每个元素占 2 bytes 的 16 位浮点格式），大小分别为 8、96、24 MiB。若 A 、 W 各读一次， C 写一次，只需传输 128 MiB。为何一种分块实现会产生约 3 GiB 的访问？先看一个输出怎样算出来。

计算 $C[i,j]$ 时，取 A 的第 i 行和 W 的第 j 列，将对应元素相乘，再沿 K 维求和。接着计算右边的 $C[i,j+1]$ ， W 换成相邻一列， A 却仍是同一行。图中两次计算的蓝色部分完全相同。

先计算一个输出



再计算相邻输出

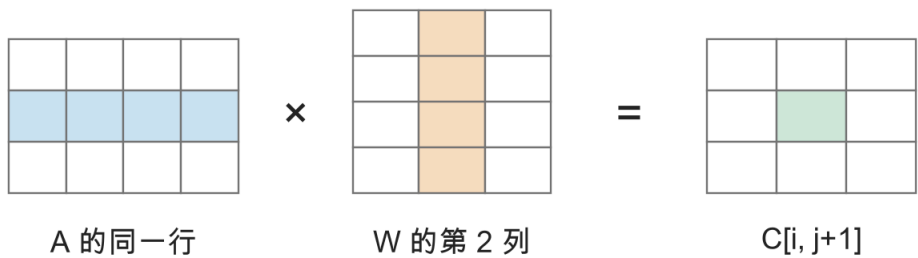


图 5-2：先算一个输出，再算相邻输出。蓝色表示所用的 A 行，橙色表示所用的 W 列，绿色表示本次得到的输出元素。图中矩阵缩小为示意尺寸，实际算例的 K 为 4096。

如果这行 A 在两次使用之间仍留在局部存储或缓存中，就能直接复用；若已被替换，就需要再次向下一层请求。转到下一行输出时， W 中的元素也会再次使用。数学上需要同一份数据多次参与计算，程序如何安排这些使用，决定了它要跨越存储接口多少次。

因此，本节先计数“工作缓冲与下一层存储之间的访问”。这不直接等于显存流量：如果下一层是 L2（二级缓存），多个线程块的重复请求可能由 L2 中的同一份副本提供，只有未命中时才继续请求显存。后面的约 3 GiB 也统计这一接口上的访问字节数。²

把图中的逐元素计算写成循环，就是：

```
for i in range(M):
    for j in range(N):
        acc = 0.0
        for k in range(K):
            acc += A[i, k] * W[k, j]
        C[i, j] = acc
```

内层 k 每走完一遍，得到一个输出。一次乘加计两个 FLOPs，全部输出的工作量为：

$$F_{\text{gemm}} = 2MKN \approx 103 \text{ GFLOPs.}$$

作为比较基准，三份 BF16 矩阵各经过接口一次的读写量为：

$$V_{\text{once}} = 2(MK + KN + MN) = (8 + 96 + 24) \text{ MiB} = 128 \text{ MiB.}$$

其中 MiB 为 2^{20} bytes。这个基准只数各输入一次读取和最终输出一次写出，并不保证给定容量下的程序能够做到。

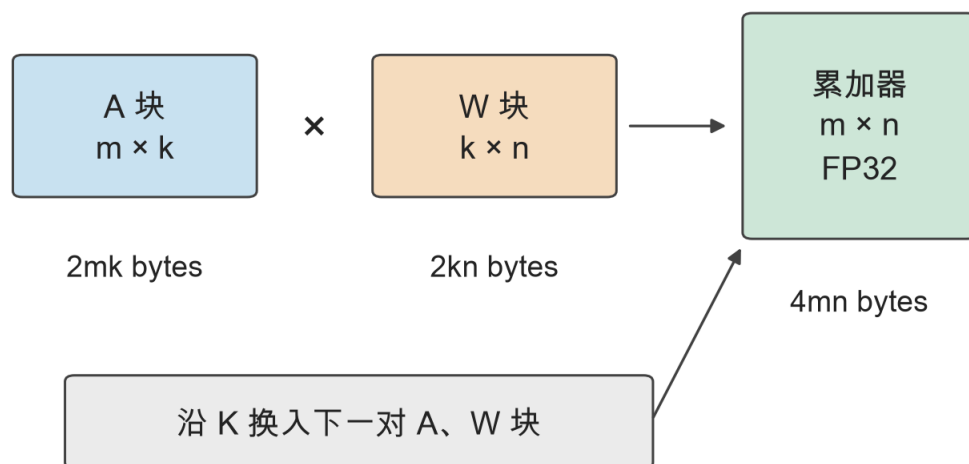
循环顺序还决定每次访问的地址。W 按行连续存储时，上面的内层循环依次读取 $W[k,j]$ 、 $W[k+1,j]$ ，地址相差 N 个元素。若改成 i、k、j 的顺序，内层便依次读取 $W[k,j]$ 、 $W[k,j+1]$ ，同时将 $A[i,k]$ 用于多个输出。这样一次连续传输取得的数据能立即参与计算。

不过，新顺序也要同时保存一段输出的累加结果。若这些结果放不进局部存储，每轮更新都可能增加向下一层的读写。接下来用分块限制同时保留的数据量，让输入复用与输出保存相互配合。

5.2.2 容量约束下的分块复用

先保留一个 $m \times n$ 输出块，每次沿 K 维读入一对输入块：A 块为 $m \times k$ ，W 块为 $k \times n$ 。它们相乘，更新同一份输出累加器；换入下一对输入块时，已经得到的部分和继续保留。直到沿 K 完成全部累加，才把结果转为 BF16 并写出。

一对输入块，更新整个输出块



输入块更换；输出累加器一直保留

图 5-3：一次块乘更新整个 $m \times n$ 输出块。A、W 输入为 BF16，累加器为 FP32（每个元素占 4 bytes 的 32 位浮点格式）；沿 K 换入新输入块时，累加器一直保留。图中尺寸符号表示形状，方框面积不按字节数缩放。

A 块中的一个元素可以用于 n 列输出，W 块中的一个元素可以用于 m 行输出。扩大输出块，就能让一次读入的数据参与更多乘加；同时也要保存更多输入和输出部分和。

例 5-3：扩大输出块能减少多少读取。图中的三个对象分别占 $2mk$ 、 $2kn$ 和 $4mn$ bytes。只使用一组输入缓冲时，局部存储需求为：

$$S = \underbrace{2mk}_{A \text{ 块}} + \underbrace{2kn}_{W \text{ 块}} + \underbrace{4mn}_{\text{累加器}} \text{ bytes.}$$

取 $k = 32$ ，先用 64×64 输出块。A 块和 W 块各占 4 KiB，累加器占 16 KiB，合计 24 KiB。把输出块扩大到 128×128 后，两份输入各占 8 KiB，累加器占 64 KiB，合计增至 80 KiB。输入复用增多，输出累加器也增长得更快。

接着计算全矩阵的读写。假设每个输出块独立读取自己的输入，不考虑不同输出块之间的缓存复用。N 方向有 N/n 个列块，每个列块都要遍历 A；M 方向有 M/m 个行块，每个行块都要遍历 W。最终 C 只写出一次，因此：

$$V = \underbrace{2MK \frac{N}{n}}_{A \text{ 的重复读取}} + \underbrace{2KN \frac{M}{m}}_{W \text{ 的重复读取}} + \underbrace{2MN}_{C \text{ 写出}}.$$

64×64 输出块将输出划为 16 个行块、192 个列块。192 个列块各读一遍 8 MiB 的 A，共 1536 MiB；16 个行块各读一遍 96 MiB 的 W，也是 1536 MiB。加上 24 MiB 输出，总量为 3096 MiB。这就是本节开头约 3 GiB 的来源。

改成 128×128 后，行块数从 16 减到 8，列块数从 192 减到 96，两份输入的读取均减半。输出大小不变，总量降至 768 + 768 + 24 = 1560 MiB。图 5-4 把这项收益与占用的局部存储放在一起比较。

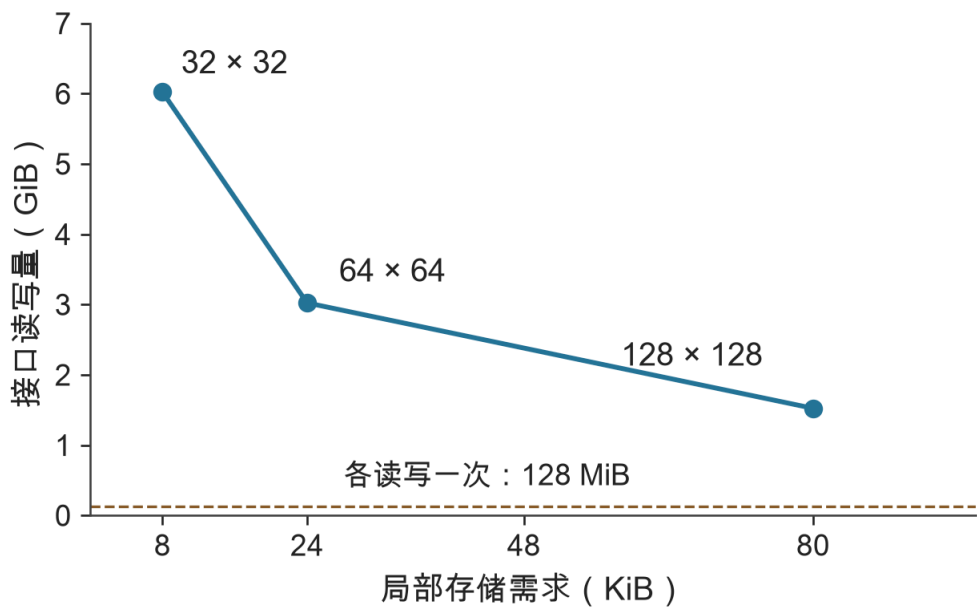


图 5-4：每个点标出输出块形状。扩大输出块能减少跨接口的重复读取，但需要更多局部存储。虚线为三份矩阵各经过一次的 128 MiB；比较条件为 k=32、单组输入缓冲、各输出块独立读取输入。

完整数值如下，算术强度用同一矩阵乘的 FLOPs 除以上述接口字节数计算：

输出块	局部存储需求	工作缓冲与下一层之间的访问	算术强度
32×32	8 KiB	6168 MiB	15.9 FLOP/byte
64×64	24 KiB	3096 MiB	31.7 FLOP/byte
128×128	80 KiB	1560 MiB	63.0 FLOP/byte

访问量接近减半，时间是否也会减半？还要看设备能同时安排多少工作。先用一个简化的容量模型：每个线程块（一组协作执行、可共享局部数据的 GPU 线程）负责一个输出块，所有工作集共用 96 KiB 的局部存储预算。

同样的 96 KiB，能放下几份工作集？

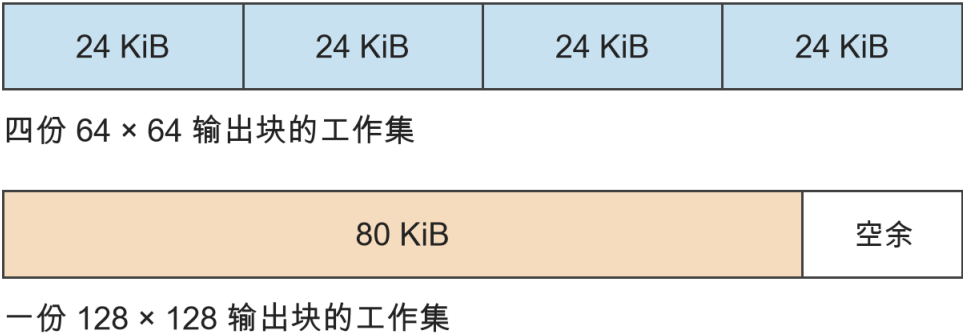


图 5-5：条形总长均表示 96 KiB。24 KiB 的工作集能放四份，80 KiB 的只能放一份。这里把输入缓冲和累加器合并计入一个教学预算；实际 GPU 的共享内存、寄存器分别受限，真实驻留数还取决于线程数等约束。

同时驻留的线程块可以交错发起访存：一些块等待数据时，另一些块继续工作。工作集从四份减为一份，可能降低隐藏访存等待的能力。因此，大块虽然少读数据，实际获得的带宽却可能下降。

假设两种实现都受这一层的带宽限制，有效带宽分别为 B_{64} 和 B_{128} ，则：

$$\frac{T_{128}}{T_{64}} = \frac{1560}{3096} \frac{B_{64}}{B_{128}}.$$

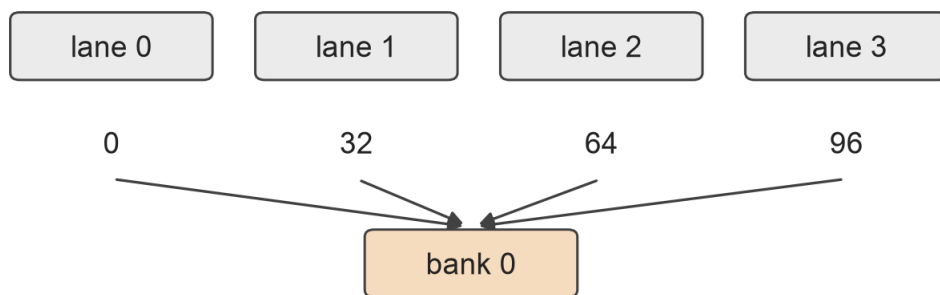
若带宽相同，时间接近减半；若大块只能获得原来四成的带宽，时间比约为 $0.504/0.4 = 1.26$ ，反而慢约 26%。可以先按容量排除放不下的方案，再按访问量缩小比较范围，最后测量实际执行效率。5.4 节将把这一过程交给编译器。

5.2.3 布局与归约的并行代价

上一小节中，大块虽然少读数据，却可能因并行度下降而变慢。并行度不仅取决于驻留多少线程块，也取决于这些线程发出的读取请求能否同时得到服务。先看局部存储的布局。考虑一个 32×32 的 FP32 暂存块，分成 32 个可独立服务读取请求的存储分体，称为 bank，每个 bank 每轮提供一个 32-bit 字，bank 号为字地址对 32 取模。这里将同时执行的一组线程中的各个位置称为 lane；32 个 lane 各读一行中的一个字时，落在 32 个 bank；读一列时，行跨度为 32 个字，所有地址落到同一 bank，需要 32 轮。

行跨度 32 个字

数字为字地址



行跨度 33 个字

数字为字地址

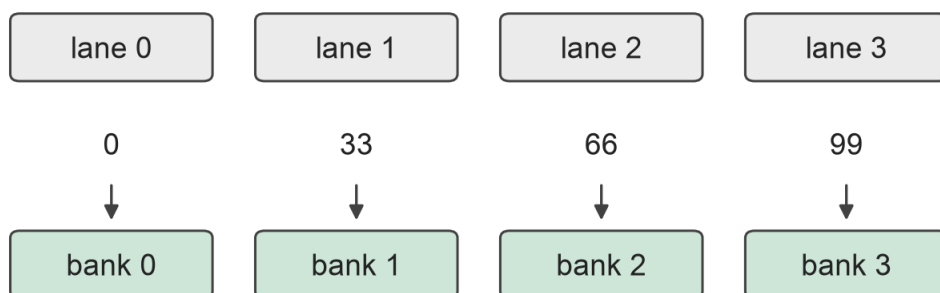


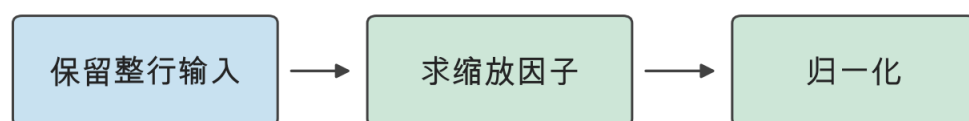
图 5-6：同一列的 32 个不同字由 32 个 lane 同时请求。上半图行跨度为 32 个字，请求集中到同一 bank；下半图补齐为 33 个字，请求分散到 32 个 bank。图中画出前四个请求；假设每个 bank 每轮提供一个 32-bit 字，不计广播。

在每行末尾补一个字，行跨度变为 33。同一列相邻行的 bank 号便依次加一，32 次请求分散到 32 个 bank，一轮即可完成。所占空间由 4096 bytes 增至 4224 bytes，只多约 3.1%。有效数据和读取次数都没有变化，但原本需要分 32 轮完成的列读取，现在可以在一轮内完成。

这种在行末增加填充元素的做法称为 padding（填充）。它让已有的并行请求分散到不同 bank。但如果程序本来就只安排了少量任务，改变地址布局还不够，还要把计算拆开。RMSNorm（均方根归一化）先根据一行元素的均方值计算共同的缩放因子，再缩放这一行。将多个元素合并为一个统计量的操作称为归约，它正好说明这种情况：一行的缩放因子来自整行平方和，随后每个元素乘以这个因子和对应的可学习参数 gamma。对 [1024, 4096] BF16 输入，一组处理一行，可以独立安排 1024 组；对单请求每次生成一个 token 的解码步骤（decode），同一种分配只产生一组，设备的大量执行资源会空闲。

把每行分成八个 512 元素片段，可以增加第一阶段的并行任务。完整归约随之变成三个阶段：每个片段产生局部平方和，合并八个局部和得到行缩放因子，再用该因子归一化输入。前两阶段只需传递很少的数，但最后一阶段要重新读取原始数据。

一组处理一行



把一行拆成八段

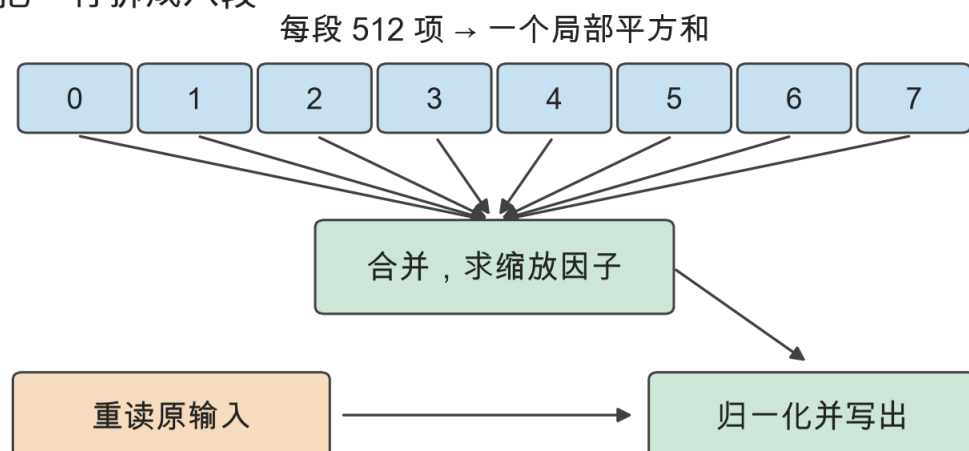


图 5-7：上行由一组处理完整一行，输入保留到归一化结束；下行把一行分为八段，先求局部和，再合并，最后重读输入并归一化。每行 4096 个 BF16 元素；图中仅画一行，推广到 1024 行时，第一阶段任务数由 1024 增至 8192。橙色框表示增加的原输入读取。

沿着图 5-7 的箭头计数，就能求出增加并行任务所付出的访存代价。仍取 1024 行，第一阶段从 1024 组增到 8192 组；8192 个 FP32 局部和占 32 KiB，1024 个 FP32 缩放因子占 4 KiB。一组一行的程序在局部保留输入，读入 8 MiB 输入，每行再读取一份 BF16 gamma，累计 8 MiB，最后写出 8 MiB 结果，访问为 $8 + 8 + 8 = 24$ MiB。拆分后，最终应用阶段重读 8 MiB 输入；局部和、缩放因子的写出与读取再增加约 0.1 MiB，总量为 32.1 MiB，kernel 启动从一次变成三次。³

图中每组只处理八分之一行，还带来另一项收益：需要保存的临时数据随之减少。以 FP32 保存完整一行需要 16 KiB，保存其中八分之一只需 2 KiB。原来因寄存器不足而写入较远存储的临时数据，可以因此保留在寄存器中。并行任务数增加与每组临时数据减少，是拆分归约获得收益的两个具体来源；增加的输入读取和阶段间数据传递则构成额外开销。

矩阵分块、bank padding 和归约拆分都在重新安排同一批数学工作。分块让多个输出复用输入，padding 将读取请求分散到不同 bank，拆分归约则增加能够并行执行的任务。分析程序的耗时，可以分别检查这些变化带来了多少收益和开销。配套实验中的连续访问与宽行归约对照提供了对应的执行记录。⁴

5.3 算子链：融合、缓冲与流水

单算子内部可以利用局部存储复用输入。相邻算子之间也有同样的机会：前一个算子的输出，往往立即成为后一个算子的输入。如果后一个算子能直接读取局部存储中的结果，就能省去一次写回和重读。为此，需要确定后一个算子要用哪些数据，以及这些数据何时全部算好。

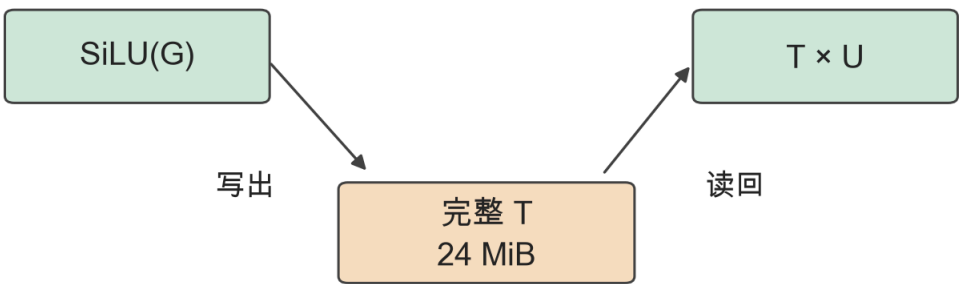
5.3.1 中间张量与融合边界

下面将算例扩展到前馈网络（FFN）的两支投影。令 $G = XW_g$ 、 $U = XW_u$ ，激活链计算 $Z = \text{SiLU}(G) \odot U$ ，再将 Z 交给 down 投影。 G 、 U 、 Z 的形状均为 $[1024, 12288]$ ，以 BF16 保存时各占 24 MiB。

这里 SiLU 是激活函数， $\text{SiLU}(x) = x/(1 + e^{-x})$ ；符号 $\odot$ 表示对应位置的元素相乘。SiLU 和乘法都是逐元素运算。 $Z[i,j]$ 只依赖 $G[i,j]$ 与 $U[i,j]$ ，不同位置之间没有归约。因此，读入同一位置的两个元素后，就能直接算出该位置的最终结果，而无需先生成全部 SiLU 结果。

先比较中间结果 T 的去向。分开执行时，完整 T 要写到下一层存储，再被乘法读回；融合后，一片 SiLU 结果可以直接交给同一 GPU 内核程序（kernel）中的乘法，用完就释放局部空间。

分开执行：中间结果经过下一层存储



融合执行：逐片传给下一步

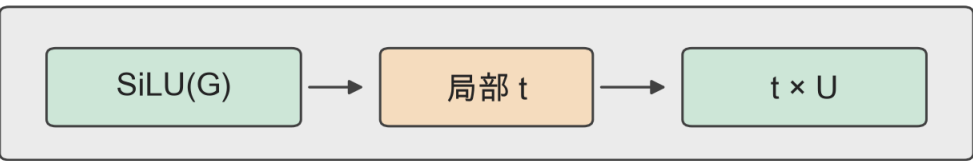


图 5-8：上半图的完整 T 经过一次写出与一次读回；下半图中局部片段 t 直接传给乘法。两种方式仍读取 G 、 U 并写出 Z ，图中省略这些共同的输入输出边。融合保持原来的中间舍入语义。

例 5-4：省去中间结果的写回和重读。分开执行时，先计算 $T = \text{SiLU}(G)$ ，读取 G、写出 T，共 48 MiB；再计算 $Z = T \odot U$ ，读取 T、U，写出 Z，共 72 MiB。总访问为 120 MiB。将两步融合，每次读取 G、U 的一片，在局部完成 SiLU 和乘法，写出 Z，只需 72 MiB。

省下的 48 MiB 正是 T 的一次写回和一次读取。沿用上一节的带宽模型，取两种程序的有效带宽相同，时间比为 $72/120 = 0.6$ ，加速约 1.7 倍。实际激活链对照从约 14.2 μs 降到 7.7 μs ，约快 1.8 倍。理论计算与实测结果都说明了同一个收益来源：省去完整 T 的写回和重读。⁶

接下来将 Z 按固定尺度量化，使每个元素只占 1 byte。独立转换需读取 24 MiB 的 Z，写出 12 MiB 量化结果，增加 36 MiB。三步完全分开的总量为 156 MiB；融合前两步为 108 MiB；三步全部融合为 $24 + 24 + 12 = 60$ MiB。图 5-9 将必需的输入输出读写画在每条柱形的左侧，再依次加上两个中间量的读写。

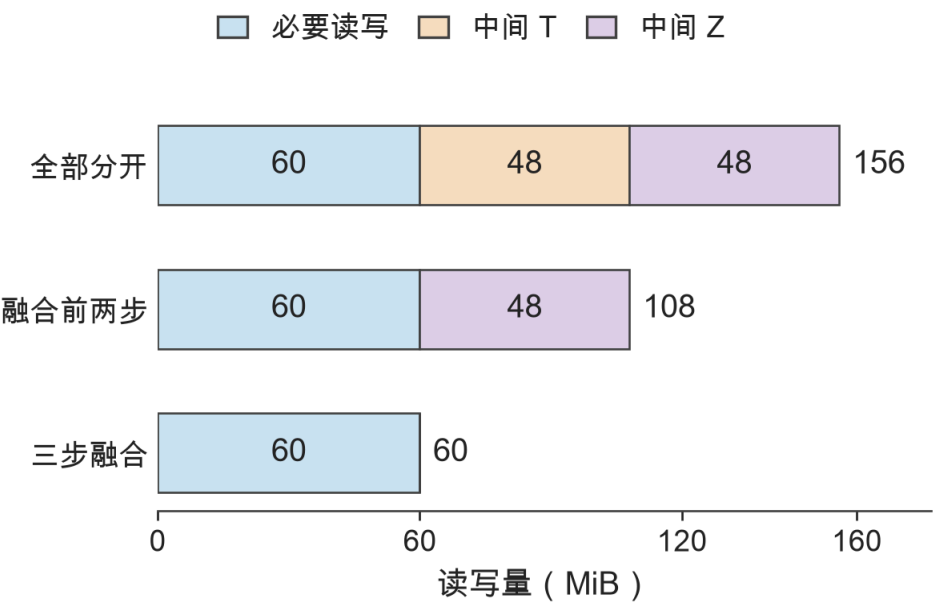


图 5-9：每少保存一个 24 MiB 中间量，就少一次写出和一次读入，共 48 MiB。G、U 为 BF16，最终输出占 1 byte/元素，量化尺度预先给定。各方案保留相同的运算与舍入顺序。

中间张量的容量与访问量在这里具有不同含义。设每步先分配输出，完成后释放已用完的输入。独立执行 SiLU 时，G、U、T 同时存活，峰值为 72 MiB；三步融合时，G、U 与最终输出同时存活，峰值为 60 MiB。读写减少 96 MiB，峰值却只减少 12 MiB，因为读写会反复经过同一份空间。⁵

逐元素运算的每个输出只依赖对应位置的输入，因此可以在读入后立即计算，减少中间结果的保存。要让这种复用顺利发生，还得让前后两步使用相容的数据布局，并保证缓冲在最后一次读取结束前不会被覆盖。下面沿着一个数据块的使用过程，分析这两个条件。

5.3.2 布局转换、缓冲复用与双缓冲

相邻算子有时使用不同布局。前一个算子适合按行写出，后一个算子却需要按另一种块顺序读取。若另建一份 24 MiB 重排结果，就要读原结果、写新结果，共 48 MiB。也可以让前一个算子直接按后续计算所需的布局写出，省去单独的重排。比较“计算后再重排”与“计算时直接按新布局写出”的总时间，就能判断哪种方式更快。

缓冲还决定两个阶段能否重叠执行。设搬运器把块 0 放入槽 A，计算单元随后读取 A。在 A 被读取期间，搬运器将块 1 放入槽 B；计算转向 B 后，A 便可装入块 2。这样，两个槽交替使用，使计算单元读取当前块时，搬运器可以写入下一块。

注意，槽 A 的“数据已经搬完”与“数据已经用完”是两个时刻。搬运结束后，计算才开始读取；只有最后一次读取结束，槽 A 才能覆盖成下一个块。取每块搬运 2 μs、计算 3 μs，搬运器和计算单元独立工作。先看三个时段的槽位状态，再把它们放回完整时间线。

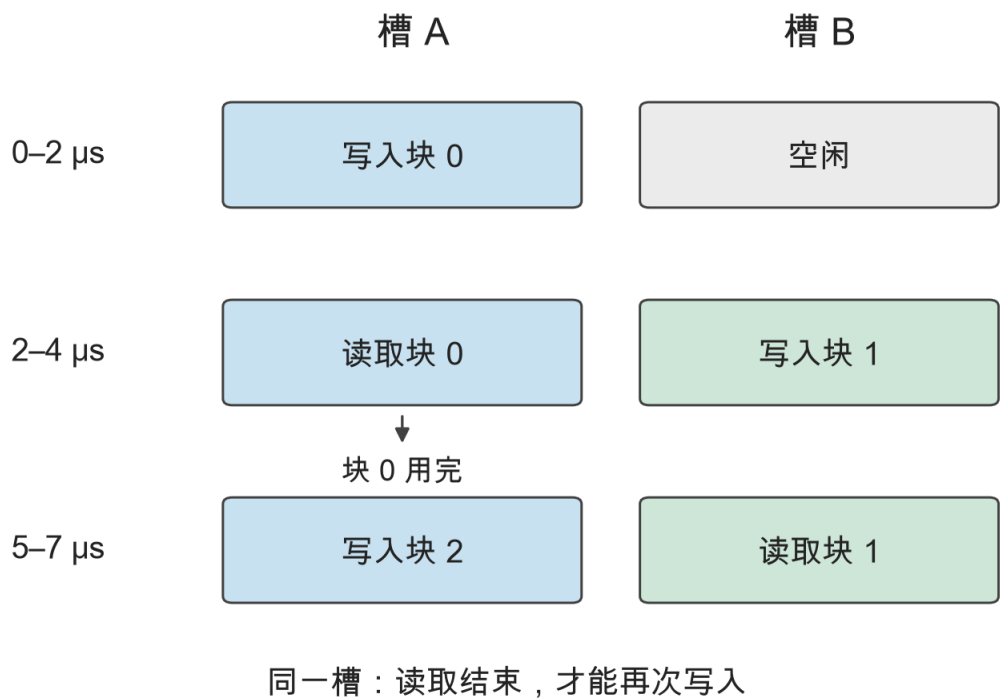


图 5-10：块 0 在 2-5 μs 使用槽 A，所以块 2 到 5 μs 才能开始写入 A。图中抽取三个时段；4-5 μs 的块 1 已在槽 B 中等待，尚未开始计算。颜色固定表示槽 A、B，文字说明当前读写的是哪个块。

例 5-5：四块数据的双缓冲。 每块搬运 2 μs，计算 3 μs。搬运器和计算单元独立工作，事件开销记为零。串行执行四块需要 $4 \times (2 + 3) = 20 \mu s$ 。

采用两个槽后，0-2 μs 搬入块 0，2-5 μs 计算块 0；块 1 在 2-4 μs 搬入，5 μs 时已经就绪。此后每隔 3 μs 完成一块。块 2 在 5 μs 才开始写槽 A，因为此前计算仍在读块 0；块 3 同理在 8 μs 开始写槽 B。第四块于 14 μs 完成，时间减少 30%。

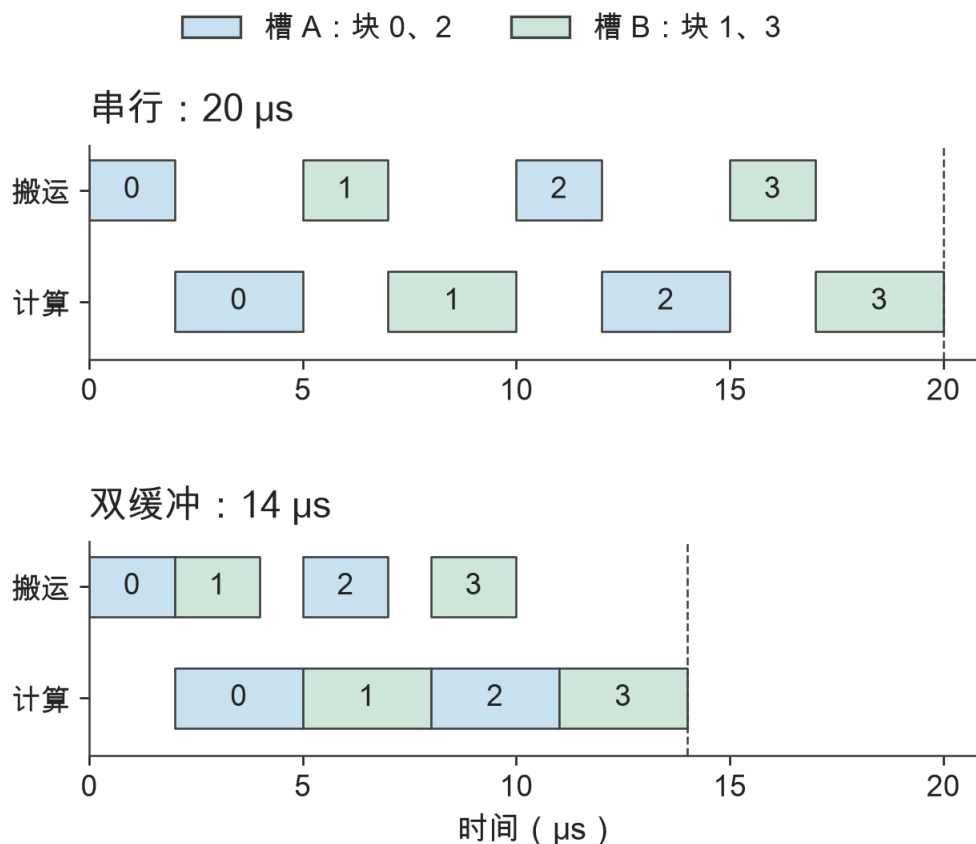


图 5-11：两个输入槽交替复用，让搬运与计算重叠。每块搬运 2 μs 、计算 3 μs ，资源独立，忽略同步开销；同一颜色表示同一槽，计算读取结束后才能再次写入。

将四块推广到 n 块。首块要先搬运，随后流水每隔两个阶段中较长的时间推出一块，末块还需完成计算。因此：

$$T_{\text{pipe}} = t_m + (n - 1) \max(t_m, t_c) + t_c.$$

其中 t_m 为每块搬运时间， t_c 为每块计算时间。在刚才的例子中，首块搬运后，计算单元连续忙了 12 μs ，总时间已是 $2 + 4 \times 3 = 14 \mu\text{s}$ 。增加第三个槽可以让后续数据更早搬入，却无法让计算单元每 3 μs 完成一块的速度再提高。此时要继续缩短时间，应减少每块计算，而不是继续增加输入缓冲。

当两阶段访问顺序不同，缓冲需要保存更多尚未使用的数据。三个排队的 16 KiB 块占 48 KiB，再加 32 KiB 重排空间，共需 80 KiB。若预算只有 64 KiB，写入第三块之前，必须等待计算单元用完并释放一个槽。这种下游来不及处理、使上游暂停的机制称为背压 (**backpressure**)。选择共同布局能够减少排队，扩大缓冲则能容纳更多已经写入但尚未使用的数据。⁷

5.1 节的主机输入也可以按此组织。这里的流 (stream) 是按提交顺序执行操作的队列，事件 (event) 用于标记进度，让另一条流等待指定操作完成。复制流传入下一批，计算流处理当前批；计算流等待“复制完成”事件后读取新输入，复制流等待“计算完成”事件后覆盖旧缓冲。这两种事件分别保证输入已准备好和旧数据已用完，时间收益则来自两种资源同时工作。

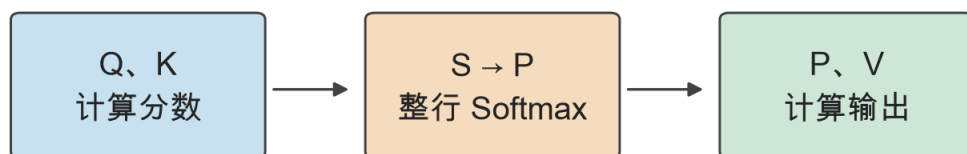
5.3.3 FlashAttention：分块与在线 Softmax

FlashAttention 由斯坦福大学的 Tri Dao 等研究者与纽约州立大学布法罗分校的合作者于 2022 年提出，针对长序列注意力中间矩阵过大、显存读写量高的问题，重新组织计算顺序与数据存储。⁹

前面的流水在一块数据准备好后就开始计算，随后释放该块缓冲。逐元素运算很容易这样处理；注意力的输出却依赖整行分数，处理完当前块时，后面的分数还没有读到。要继续沿用逐块计算和复用缓冲的方式，就必须找到能够正确合并各块结果的归约方法。其中 Q 是查询向量， K 是供查询匹配的键向量， V 是最终按权重汇总的值向量。Softmax 将一行分数转换为总和为 1 的权重。标准注意力先计算 $S = QK^T/\sqrt{d}$ ，再计算 $P = \text{softmax}(S)$ ，最后计算 $O = PV$ 。

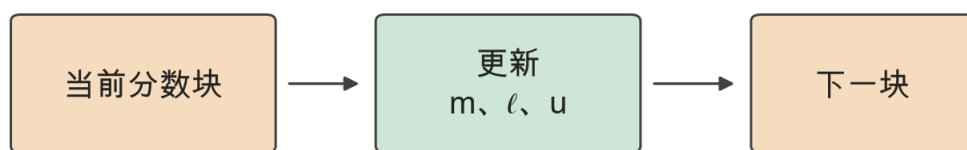
取一个头，序列长度 $L = 8192$ ，头维度 $d = 128$ ，对全部位置计算注意力。 Q 、 K 、 V 、 O 使用 BF16，各占 2 MiB；完整 S 、 P 使用 FP32，各占 256 MiB。 Q 、 K 、 V 各读一次， O 写一次，共 8 MiB； S 、 P 各写一次、读一次，却要传输 1 GiB 数据。中间矩阵的大小随序列长度的平方增长，因而造成了大量读写。⁸

保存完整中间矩阵



S 、 P 各 256 MiB，分别写出再读回

处理当前块，留下统计量



传递统计量；分数缓冲留给下一块

图 5-12：上半图保存完整 S 、 P ，两份 FP32 矩阵各占 256 MiB，写出与读回合计 1 GiB。下半图只传递已处理部分的最大值 m 、指数和 l 、加权值 u ；每处理完一个块，其分数缓冲即可复用。箭头概括处理顺序，完整计算还需读入 Q 、 K 、 V 。

直接套用逐元素融合会遇到一个障碍：Softmax 的分母是整行指数和。只读到前一块时，既不知道最终分母，也不知道后面是否会出现更大的分数。要在处理完一个块后丢弃其中的分数，就需要保留几个统计量，使后续计算仍能计入这个块对结果的贡献。

要留下什么，才能忘掉旧分数？对于已经读过的位置，保留三个量：最大分数 m ，以 m 为基准的指数和 ℓ ，以及相同指数权重下的值之和 u 。这里 u 通常是向量，先用标量值的小例子看清更新过程。

例 5-6：用两个块完成一行 **Softmax**。分数为 $[0, \ln 2]$ ，对应值为 $[1, 3]$ ，每块只含一个元素。第一块得到 $m = 0, \ell = 1, u = 1$ 。处理第二块后，最大值增至 $\ln 2$ ，将原来的 ℓ 和 u 都乘以 $r = 1/2$ ；新元素的指数为 1，于是 $\ell' = 1/2 + 1 = 1.5$ ， $u' = 1/2 + 3 = 3.5$ ，结果为 $7/3$ 。一次处理整行时，两个权重正比于 1 和 2，结果同样为 $(1 + 2 \times 3)/3 = 7/3$ 。

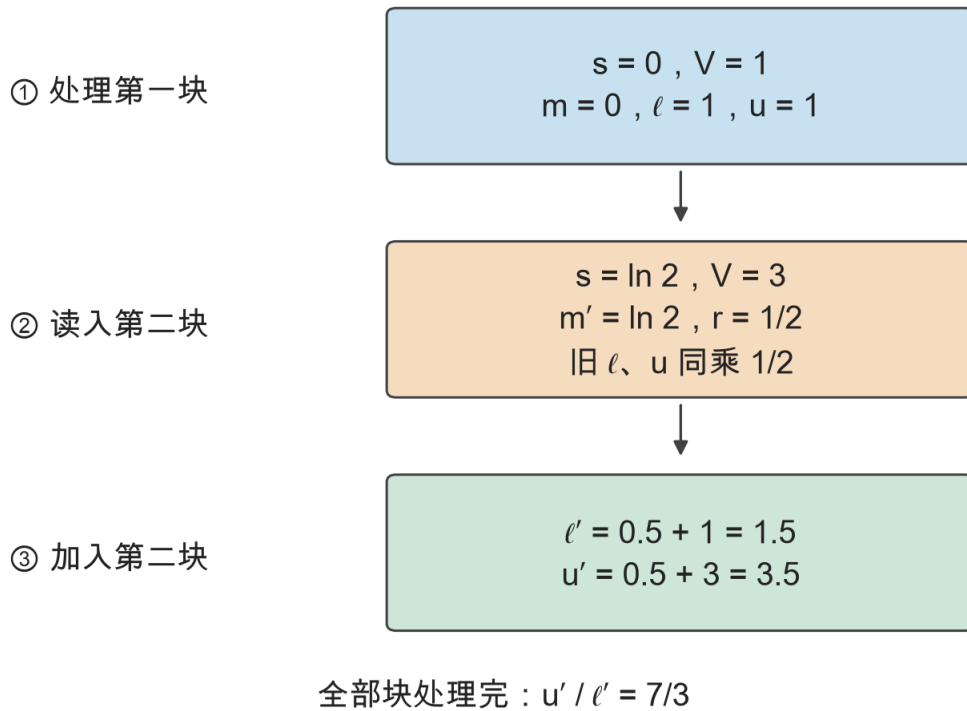


图 5-13：两个块的分数分别为 0、 $\ln 2$ ，值分别为 1、3。最大值增大后，将旧指数和与旧加权值同时乘以 $1/2$ ，再加上新块的贡献，最后才做除法。箭头传递的是统计量，旧分数无需保留。

小例子中，第二块把最大值从 0 提高到 $\ln 2$ 。旧项原来的指数为 1，换到新基准后变成 $1/2$ ；分母中的旧贡献和分子中的旧贡献必须同时缩小，才能保留它们的相对权重。

推广到一行分数，减去共同最大值可避免指数溢出。令 $\ell = \sum_j \exp(s_j - m)$ 、 $u = \sum_j \exp(s_j - m)V_j$ ，最终输出为 u/ℓ 。新块到来后，把旧状态改到新的最大值基准，再加上新块贡献：

$$m' = \max(m, \max(s)), \quad r = \exp(m - m'), \quad p_j = \exp(s_j - m'),$$

$$\ell' = r\ell + \sum_j p_j, \quad u' = ru + \sum_j p_j V_j.$$

新块中的 j 才出现在右侧求和里，旧块的全部贡献已经保存在 ℓ 与 u 中，无需重新读回旧分数。

FlashAttention 正是利用这一递推关系省去了完整的 S 和 P。每处理完一块，就更新 m 、 ℓ 和 u ；该块的分数和概率用完即可丢弃，其缓冲可以留给下一块。只保存这些统计量，就能继续计算后续块。

图 5-13 中，旧块留下的是三个统计量，分数缓冲可以反复使用。这样就把存储完整分数矩阵的问题，变成了在快速存储中安排当前块的问题。接下来要确定：一次处理多少行 Q、多少行 K/V，才能更好地利用这块空间。设快速缓冲为 128 KiB，一次保留 a 行 Q 与 FP32 输出累加器，分数块大小为 $a \times b$ 。K 与 V 按阶段复用同一个 $b \times d$ 输入槽，分数和概率复用 $a \times b$ 槽，另为逐行统计和更新暂存分配三个 FP32 向量。局部存储需求为：

$$S = \underbrace{2ad}_Q + \underbrace{2bd}_{K/V \text{ 槽}} + \underbrace{4ab}_{\text{分数/概率}} + \underbrace{4ad}_{\text{输出累加器}} + \underbrace{12a}_{\text{逐行状态}} \leq 131072.$$

K/V 块越小，它自身和分数块占用越少，就能放下更多 Q 行。更多 Q 行共享一次 K/V 扫描，整份 K/V 被重读的次数随之减少。取 $b=64$ ，能放下 $a=110$ 行 Q，需要 $\lceil 8192/110 \rceil = 75$ 次完整扫描。每次读 4 MiB 的 K/V，另有一次 Q 读取和 O 写回，共 $75 \times 4 + 4 = 304$ MiB。

将 b 缩为 1，能放下 $a=166$ ，扫描次数降为 50，访问降至 $50 \times 4 + 4 = 204$ MiB。但每次扫描从 128 个 K/V 块变成 8192 个块，在线状态必须更频繁地更新。

K/V 块行数 b	可保留的 Q 行数 a	K/V 完整扫描次数	缓冲与下一层之间的访问	块对更新次数
1	166	50	204 MiB	409600
64	110	75	304 MiB	9600
128	76	108	436 MiB	6912

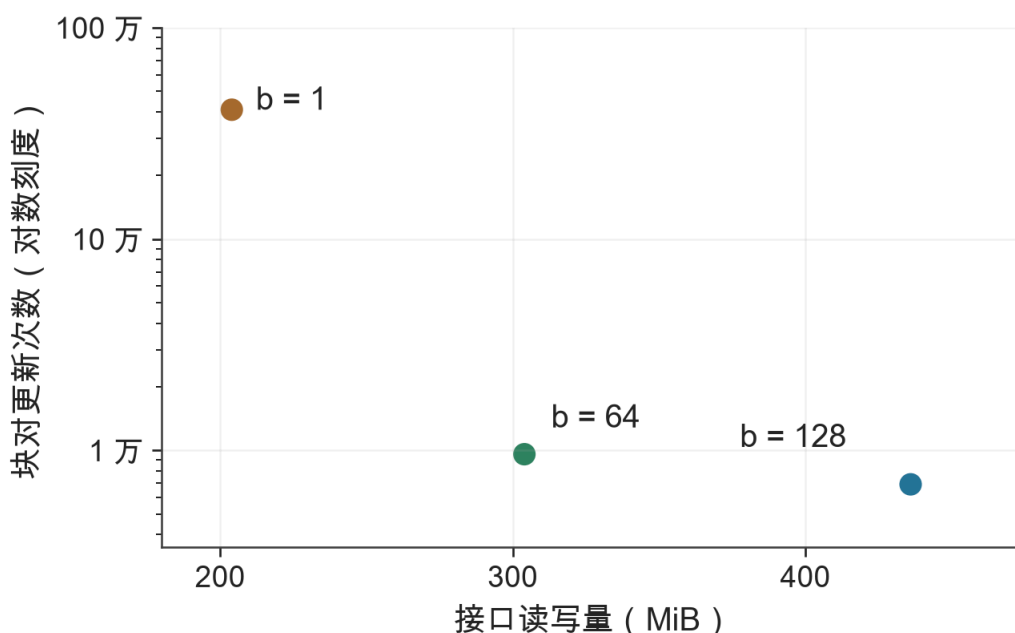


图 5-14：快速缓冲为 128 KiB，序列长 8192、头维度 128，无掩码；每个点对应正文表格的一种 K/V 块大小。横轴为缓冲与下一层之间的访问量，纵轴为在线更新次数，采用对数刻度。从 b=64 的点移到 b=1 的点，读取减少，更新次数却约增至 43 倍。

图 5-14 把两项代价放在同一平面上，越靠左表示少读数据，越靠下表示少做更新。从 64 行块缩到 1 行块，少读 100 MiB，却把块对更新从 9600 次增至 409600 次，约为原来的 43 倍。每次更新都要执行循环控制、最大值比较、指数与输出缩放；每次只处理一行 K/V，也使矩阵乘的一个维度过小，难以充分利用矩阵计算单元。较小的 K/V 块能减少读取，却会增加更新次数；较大的块则有利于矩阵计算，并减少循环开销。因此，选择块大小时，需要同时比较这两类开销。

在线归约消除了完整中间矩阵的读写，但块内的矩阵乘、指数计算和统计量更新仍要执行。后续优化便转向这些剩余工作。FlashAttention-2 改进了线程之间的任务分配，减少中间数据交换和非矩阵运算；FlashAttention-3 利用异步执行，使搬运、矩阵乘和 Softmax 重叠进行。省去巨大的中间矩阵之后，这些改进进一步缩短了剩余的计算和搬运时间。⁹

为了检验这些机制在实际程序中的效果，配套实验比较了 PyTorch 中同一个注意力接口的两种执行实现。这里的后端 (**backend**)，指框架在接口内部实际选用的计算程序：调用者给出相同的 Q、K、V，框架可以用不同程序完成注意力计算。

数学后端 (`SDPBackend.MATH`) 按注意力公式组合矩阵乘、Softmax 等通用运算，保存完整的分数和概率中间矩阵。**FlashAttention** 后端 (`SDPBackend.FLASH_ATTENTION`) 采用前面介绍的分块与在线归约方法，避免保存完整的中间矩阵。实验强制选择其中一种实现，以确保比较的是指定的两条执行路径；这里的“数学”是 PyTorch 的实现名称，两种实现计算的都是注意力。

实验在 RTX PRO 6000 Blackwell 上进行，输入输出为 BF16，一次处理一个序列、一个注意力头，头维度为 128。序列长为 8192 token，其中 token 是模型处理的一个离散输入位置；采用因果注意力，即每个位置只使用自身及此前的位置。这里的因果条件与前面无掩码的容量算例不同。预热后，使用 CUDA Graph 将一组 GPU 操作预先记录并重复执行，以减少逐次由主机提交的干扰。按每次注意力计算统计，保存完整中间矩阵的数学实现约需 2.6 ms，采用分块与在线归约的 FlashAttention 实现约需 80 μ s，后者约快 33 倍。¹⁰

另外，在输入 Q、K、V 已驻留显存的条件下，测量一次普通调用新分配的显存峰值，包含输出与临时工作区：数学实现约为 848 MiB，FlashAttention 实现约为 22 MiB。这不是整个模型或进程的显存占用。两条执行路径的中间精度和任务组织也有差别，33 倍是这组条件下完整实现的对比结果，不能全部归因于少写回某一份矩阵。这个例子展示了分块、在线归约和融合如何配合：分块让当前数据放得进快速存储，在线归约使 Softmax 可以逐块计算，融合省去块内中间结果的写回。

-
2. [分块容量与数据复用](#)，含循环计数、Orojenesis 选读与 bank 映射。↩
 3. [RMSNorm 三阶段拆分计算](#)。输入和 gamma 为 BF16，gamma 每行读取；一组一行保留输入，三阶段方案在应用阶段重读输入。↩
 4. [实验 5-1、GPU 分块、归约拆分](#)。↩
 5. [固定尺度融合与张量生命周期](#)。↩
 6. [实验 5-2 的原始条件、数据与分析](#)。RTX PRO 6000、共享 GPU、热缓存与图重放；原始中位数为 14.20 / 7.68 μ s，仅测激活子链。↩
 7. [流式顺序与缓冲预算](#)；[主机搬运与缓冲寿命](#)。↩
 8. [注意力分块与在线状态](#)。容量模型采用单头、无掩码、K/V 复用输入槽和 S/P 复用临时槽。↩
 9. [FlashAttention](#)、[FlashAttention-2](#)、[FlashAttention-3](#)。↩↩
 10. [实验 5-3](#)。RTX 热缓存单头对照：2.64 ms / 79.5 μ s；新增分配峰值约 848 / 22.2 MiB，包括各后端分配的临时量。↩